

Master Architecture & Project Specification

Academic Publishing, Education &
Institutional Services Platform

Document Version: 1.0.0 (Final Draft)

Date: April 2026

Classification: Internal Engineering & Architecture Blueprint

Table of Contents

- 1. Executive Business Overview & Strategic Objectives**
- 2. Platform Architecture & Infrastructure**
- 3. Core Module: Manuscript Tracking System (MTS)**
- 4. Institutional B2B Hub & Profile Aggregation**
- 5. E-Commerce & Manual Verification Engine**
- 6. Media, CMS & SEO Strategy**
- 7. Database Modeling & State Management**
- 8. Comprehensive 9-Phase Execution Roadmap**
- 9. Security Protocols & Toolchain References**

1. Executive Business Overview & Strategic Objectives

1.1 The Industry Context

The academic publishing industry is currently undergoing a massive structural shift. Traditional legacy publishers operate on antiquated, fragmented software systems that isolate the peer review process from post-publication dissemination and educational services. Furthermore, the rise of Open Access (OA) publishing has shifted the financial burden from the reader (via paywalls) to the author (via Article Processing Charges, or APCs).

This project aims to disrupt the fragmented status quo by building a unified, highly performant digital ecosystem. By bringing the Manuscript Tracking System (MTS), Institutional Memberships, Learning Management Systems (LMS), and Public Archives under one decoupled architectural roof, we drastically reduce operational overhead and provide a seamless experience for researchers, editors, and students.

1.2 The Multi-Tiered Monetization Strategy

A critical component of this project's viability is its monetization strategy. Rather than relying solely on high-volume, low-margin display advertising, the platform leverages high-value B2B and B2C transactions:

- **Institutional B2B Memberships:** The primary revenue driver. Universities and research organizations pay a significant annual flat fee. In exchange, the system automatically grants publishing waivers (APC exemptions) to any user registering with that institution's domain or invite code.
- **Article Processing Charges (APCs):** For unaffiliated authors, the platform charges a standard APC upon the successful peer review and acceptance of their manuscript. Draft and submission phases remain free to encourage volume.
- **Premium Digital Assets (B2C):** The platform acts as a digital storefront for high-end academic resources, including specialized video courses (LMS) and premium digital magazine issues.

- **Professional Services:** A dedicated portal allowing non-native English speakers or novice researchers to purchase offline academic translation, copyediting, and peer-review formatting services.

1.3 The Manual Verification Paradigm

To maximize revenue retention and cater to global academic markets where corporate credit cards are scarce (and direct wire transfers are standard), the platform explicitly avoids automated third-party payment gateways like Stripe or PayPal in its primary phase.

We are implementing a **Manual Verification Engine**. When a user wishes to pay an APC or purchase a course, the system generates an invoice with corporate bank details. The user performs an offline wire transfer and uploads the PDF/JPG receipt to the platform. An administrator visually verifies the deposit and manually triggers an "Unlock" event in the database, granting the user access to their digital goods or advancing their manuscript to publication.

2. Platform Architecture & Infrastructure

2.1 The Decoupled (Headless) Paradigm

To ensure enterprise-grade scalability and maximum SEO indexing, the system utilizes a headless, dual-repository architecture. The backend and frontend do not share the same codebase, communicating strictly via a RESTful API. This future-proofs the platform, allowing a native iOS or Android mobile application to be built later using the exact same backend engine.

2.2 The Backend Engine: Laravel 13

The backend is powered by Laravel 13 (running on PHP 8.3). It acts purely as a stateless API layer. It manages relational data integrity, complex state-machine transitions for manuscript workflows, and asynchronous background jobs. It is secured via Laravel Sanctum token authentication, ensuring that the frontend Single Page Application (SPA) components securely maintain user sessions without vulnerable cookie storage.

2.3 The Frontend Interface: Next.js & React

The frontend is a Next.js application utilizing the modern App Router and Tailwind CSS. It is entirely written in strict TypeScript. The frontend is cleanly divided into two rendering strategies:

- **Public Portals (Server-Side Rendering):** All public-facing pages—such as Journal Archives, single Article views, Blogs, and the Magazine Storefront—are rendered on the server (SSR). This ensures that when Google Scholar or standard web crawlers index the site, they instantly parse the full HTML content, abstracts, and citations.
- **Private Dashboards (Client-Side Rendering):** Once a user logs in (Author, Editor, Reviewer), they are routed to highly interactive React client components. This allows for seamless drag-and-drop file uploads, Kanban board triage, and dynamic form evaluations without page reloads.

2.4 Infrastructure Setup: KVM-2 VPS & Nginx

The application is hosted on a Hostinger KVM-2 Virtual Private Server (VPS) utilizing a LEMP stack alongside Node.js. Nginx acts as the master reverse proxy router. When a request hits the server, Nginx evaluates the subdomain or route:

- Requests to `api.domain.com` are routed to the PHP-FPM socket to be processed by Laravel.
- Requests to `www.domain.com` are proxied directly to the local Node.js PM2 daemon running the Next.js production build on port 3000.

2.5 Cloud Storage Integration: AWS S3

Academic platforms handle massive files (20MB+ PDFs, High-Resolution Cover Art, Video Courses). If these files were uploaded directly to the KVM-2 server, the bandwidth would bottleneck, and the 50GB disk drive would rapidly fill, crashing the databases. To prevent this, the architecture mandates AWS S3 integration.

The Pre-Signed URL Workflow: The frontend never sends a manuscript file to the Laravel API. Instead, the frontend requests permission to upload. Laravel verifies the user's authentication and generates a temporary, cryptographically signed AWS URL. The Next.js frontend then executes an HTTP `PUT` request, uploading the massive file directly from the user's browser to Amazon's servers. Once successful, Next.js simply tells Laravel the file's final S3 path.

3. Core Module: Manuscript Tracking System (MTS)

3.1 Overview of the MTS

The Manuscript Tracking System is the central nervous system of the platform. It governs the lifecycle of an academic paper from initial drafting to final publication. It relies heavily on Role-Based Access Control (RBAC) to ensure strict double-blind peer review principles, guaranteeing that authors and reviewers remain anonymous to one another.

3.2 The State Machine Engine

The MTS operates on a rigid programmatic state machine. A manuscript is not simply a row in a database with a changeable text field; it is an entity bound by transition rules. The Laravel backend enforces the following linear progression:

1. **Draft:** The author is currently building the submission. Fields are nullable.
2. **Submitted:** The author locks the manuscript. The system validates that all required metadata, abstracts, and S3 file paths are present.
3. **Technical Triage:** The manuscript hits the Editor-in-Chief's Kanban board. The Editor reviews the automated Copyleaks Plagiarism report. They can either issue a "Desk Reject" or advance the paper.
4. **Under Evaluation:** The Editor assigns the paper to minimum of two Reviewers. The system locks the paper, preventing the author from making further edits.
5. **Revisions Required:** Based on Reviewer scorecards, the Editor requests changes. The author is temporarily granted edit access to upload a revised S3 PDF.
6. **Accepted:** The final version is approved. The system automatically generates the APC invoice via the Manual Ledger.
7. **Published:** Upon payment verification, the system formally mints the Article, assigning it to a Journal, Volume, and Issue, and moving it to the public Archives.

3.3 Reviewer Routing & Scorecards

Editors possess access to a searchable database of registered academics, filterable by expertise and past review performance. When an Editor selects a Reviewer, the Laravel backend queues an asynchronous email invitation via the Resend API. This email contains a secure, tokenized link.

If the Reviewer accepts, they gain temporary access to a secure dashboard where they can download the manuscript PDF. They evaluate the paper using a dynamic React form, generating a structured JSON scorecard evaluating methodology, originality, and citation accuracy. The system strictly prevents assigning a Reviewer who shares a matching email domain with the Author to prevent conflicts of interest.

3.4 Automated Integrity Scanning (Copyleaks)

To combat the rise of AI-generated academic submissions, the MTS integrates directly with the Copyleaks API. Upon successful transition to the "Submitted" state, a Laravel background job dispatches the manuscript's S3 link to Copyleaks. The AI processes the text and returns a granular similarity and AI-probability score, which is securely embedded into the Editor's Triage Dashboard.

4. Institutional B2B Hub & Profile Aggregation

4.1 The B2B Value Proposition

Universities are eager to showcase the research output of their faculty and student body. The Institutional Hub serves both as a lucrative monetization channel and an automated portfolio generator for these organizations.

4.2 Organization Registration & Invite Codes

When a University purchases an annual membership, an Admin provisions an Institutional Profile. The Laravel API generates a unique, cryptographically secure Invite Code. The University distributes this code to its student body.

When a new user registers on the platform (or updates their existing profile), they can input this code. The system verifies the code and securely links the user's ID to the Institution's ID via a many-to-many pivot table.

4.3 Automated APC Waivers

The billing engine contains middleware that intercepts APC invoice generation. If the system detects that an Author is successfully linked to an active Institution, the invoice is automatically zeroed out, and the manuscript proceeds directly to the "Published" state, fulfilling the B2B contractual agreement.

4.4 SEO-Optimized Public Profiles

The Next.js frontend dynamically generates a Server-Side Rendered (SSR) profile page for every registered Institution (e.g., `/institutions/harvard-university`). This page queries the database for all published articles where the Author ID matches the Institution ID. The result is a highly visible, automatically updating feed of the University's aggregate research output, driving significant organic traffic to the platform.

5. E-Commerce & Manual Verification Engine

5.1 The Rationale for Offline Commerce

Integrating global payment gateways like Stripe introduces significant regulatory overhead, API complexity, and percentage-based transaction fees. For high-ticket B2B transactions (e.g., a \$5,000 annual university membership or a \$300 APC), retaining 100% of the margin via direct bank transfer is a strategic imperative during the platform's growth phase.

5.2 The User Checkout Journey

Whether purchasing a premium video Course, a digital Magazine, or paying an APC, the user enters a unified Next.js checkout flow:

1. The user confirms the digital item and price.
2. The frontend displays the platform's official corporate wire-transfer details, SWIFT codes, and localized banking instructions.
3. The user performs the transfer via their personal banking application.
4. The user returns to the platform and utilizes a drag-and-drop interface to upload their transaction receipt (PDF, JPG, PNG).
5. The frontend pushes the receipt directly to an isolated, private AWS S3 bucket.
6. The Laravel API creates an `Order` record, attaches the S3 receipt path, and flags the order status as `Pending Verification`.

5.3 The Admin Ledger & Fulfillment

Platform Administrators have access to a dedicated backend React dashboard called the Manual Ledger. This interface lists all pending orders chronologically. The Admin can click an order, visually inspect the S3 receipt image in a modal window, and cross-reference the deposit with the company's actual bank account.

Upon clicking the "Approve" button, the Laravel API updates the order status to `Active`. This single database mutation automatically unlocks the video course in the LMS, enables the PDF download button in the Magazine Storefront, or advances the manuscript to

publication. A background job instantly fires an email to the user confirming successful fulfillment.

6. Media, CMS & SEO Strategy

6.1 The Importance of Indexing

An academic publishing platform lives and dies by its discoverability. Researchers will not submit to journals that are not indexed by Google Scholar, Scopus, or Web of Science. These academic crawlers are notoriously poor at executing JavaScript to read Client-Side Rendered (CSR) applications. Therefore, all public portals must be Server-Side Rendered (SSR).

6.2 Next.js SSR Implementation

The Next.js App Router fetches data from the Laravel API on the server before the page is sent to the browser. When a crawler hits `/journal/v1/issue-2/article-104`, it receives raw HTML containing the full abstract, author affiliations, and DOI citations. Furthermore, dynamic `generateMetadata` functions inject specific OpenGraph and Twitter tags into the document head, ensuring highly visible link previews when articles are shared on social media or Slack.

6.3 The Magazine Storefront & Blogging Engine

To drive top-of-funnel organic traffic, the platform includes a robust Content Management System (CMS).

- **Blogging:** Admins utilize a rich-text WYSIWYG editor (TipTap) to author academic guidelines, news, and podcast summaries.
- **Events Portal:** A chronological directory of upcoming academic conferences, Call for Papers (CFPs), and registration deadlines, acting as a valuable resource for the academic community.
- **Magazine Storefront:** A visually distinct, premium UI featuring CSS Grid layouts of digital magazine covers. Unauthenticated users can view the cover art and table of contents to drive interest, but the actual PDF retrieval endpoint is protected by the Manual Verification Engine middleware.

7. Database Modeling & State Management

7.1 Relational Architecture

The system relies on MySQL 8 to manage complex, highly normalized hierarchical data. The core academic hierarchy enforces strict foreign keys:

```
Journals → Volumes (hasMany) → Issues (hasMany) → Articles (hasMany)
```

Cascade deletes are strictly disabled for these core tables. An Admin cannot accidentally delete a Journal if it contains published Articles, ensuring historical preservation and citation integrity.

7.2 The Immutable Audit Trail (status_logs)

In legacy systems, moving a manuscript from "Under Review" to "Accepted" simply overwrites a `status` column in the database, destroying the historical context. This platform utilizes an immutable audit architecture.

Every manuscript has a one-to-many relationship with a `status_logs` table. When a transition occurs, the Laravel API inserts a new row detailing the timestamp, the previous state, the new state, the user ID who initiated the change, and an optional JSON payload (e.g., capturing the exact reviewer IDs assigned during the transition). This guarantees a flawless, auditable history of the peer review process.

7.3 Server Memory Tuning

Running Node.js, PHP-FPM, and MySQL on an 8GB KVM-2 server requires strict memory balancing to prevent Linux Out-Of-Memory (OOM) killer terminations. The `my.cnf` is manually tuned to restrict the InnoDB buffer pool to ~1.5GB to 2GB, leaving ample headroom for Next.js SSR processes to spike during high-traffic events.

8. Comprehensive 9-Phase Execution Roadmap

The development lifecycle is sequenced to establish infrastructural stability before introducing complex business logic. The strategy prioritizes public-facing SEO modules first to begin building domain authority while the internal MTS is constructed.

Phase	Focus Area	Technical Objectives & Deliverables
Phase 1	Infrastructure & CI/CD	Provision Hostinger KVM-2 VPS. Configure Nginx reverse proxy blocks. Establish isolated `academic_staging` and `academic_prod` MySQL databases. Create GitHub Actions for zero-downtime SSH/Rsync deployments of both Laravel and Next.js repositories. Provision AWS S3 IAM credentials.
Phase 2	Database & API Core	Initialize Laravel 13. Configure Sanctum token authentication. Write extensive database migrations establishing the relational hierarchy, foreign keys, and the immutable `status_logs` audit trail. Implement Spatie Role-Based Access Control (RBAC).
Phase 3	Frontend Alignment	Initialize Next.js App Router and Tailwind CSS. Implement `openapi-typescript` script to dynamically pull the API schema from Laravel, ensuring strict type safety. Build the centralized Axios API client with global 401 Unauthorized interceptors.
Phase 4	Media & CMS Portals	Build Laravel CRUD endpoints for Pages, Blogs, Events, and Magazines. Develop the React Admin WYSIWYG editor. Construct the public-facing, Server-Side Rendered Next.js catch-all routes to guarantee optimal SEO indexing by web crawlers.
Phase 5	Institutional Hubs	Develop the B2B registration flow and automated invite-code generator. Build Laravel endpoints utilizing deep Eager Loading to resolve nested Journal hierarchies under 500ms. Build public University Profile pages aggregating associated student publications.

Phase 6	E-Commerce Ledger	Architect the Course LMS API and structure. Build the frontend checkout flow displaying wire transfer details and the drag-and-drop S3 receipt upload dropzone. Develop the Admin Ledger dashboard to visually inspect receipts and manually unlock digital products.
Phase 7	MTS Core Engine	Code the rigid state machine logic in Laravel, preventing unauthorized state skipping. Implement the secure AWS Pre-Signed URL endpoint. Develop the multi-step React wizard for authors to submit metadata, co-authors, and execute direct-to-S3 manuscript uploads.
Phase 8	MTS Triage & Dashboards	Build the Editor's Kanban triage board. Implement searchable Reviewer assignment with tokenized email routing. Construct secure Reviewer scorecards. Write asynchronous Laravel Jobs to interface with the Copyleaks AI API for automated plagiarism scanning upon submission.
Phase 9	Deployment & Polish	Implement cross-table global search capabilities. Program dynamic Next.js `generateMetadata` functions for OpenGraph social sharing. Execute the production rollout, merge code to the `main` branch, and lock the domains behind Cloudflare SSL certificates.

9. Security Protocols & Toolchain References

9.1 Cross-Repository API Alignment

A major risk in headless architectures is "API Drift," where the frontend expects data formats the backend no longer provides. This project entirely eliminates this risk. Laravel utilizes the `scramble` package to automatically read codebase controllers and expose a live OpenAPI specification (`api.json`). The Next.js frontend utilizes an NPM script to fetch this JSON file and compile it into strict TypeScript definitions (`schema.d.ts`). If a backend developer renames a database column, the Next.js CI/CD pipeline will instantly fail its build, guaranteeing perfect structural alignment.

9.2 Core Toolchain & Documentation Links

The following primary technologies and external services constitute the operational stack. Developers should reference these official documentations for specific integration guidelines.

Technology / Service	Role in Architecture	Official Documentation
Laravel 13 (PHP 8.3)	Backend REST API, State Machine, Auth	https://laravel.com/docs
Next.js (App Router)	Frontend UI, SSR, Routing	https://nextjs.org/docs
Dedoc Scramble	OpenAPI Generation for API alignment	https://scramble.dedoc.co/
Copyleaks API	Automated AI & Plagiarism scanning	https://api.copyleaks.com/documentation
Resend API	Asynchronous Transactional Email Queues	https://resend.com/docs
AWS S3 Pre-Signed URLs	Direct-to-Cloud frontend file uploads	AWS S3 Pre-Signed URL Documentation

Document End: This specification represents the finalized, highly detailed architectural blueprint for the Academic Publishing, Education & Institutional Services Platform. Adherence to these structural, database, and phasing guidelines is mandatory for all engineering and QA personnel moving forward.